

> RecApparō Specsheel — v0.2
> by @Voxvoltera | development@voxvoltera.com
> last updated: 22/06/2026
> status: draft

0 Revision history

This document is versioned independently of the code. Bump the version on any change to a spec entry and log it here so the Kanban board and the spec never drift apart.

Version	Date	Author	Notes
0.1	19/06/2026	@Voxvoltera	Initial draft
0.2	22/06/2026	@Voxvoltera	Local/server architecture

1 Introduction

1.1 Purpose

RecApparō is a kanban-style work-tracking application. This document is the single source of truth for both frontend and backend development — the data model, the API surface, the permission rules, and the behaviour of every operation. Implementation follows the entries here; any deviation is agreed with the maintainers and written back into this document before it ships.

1.2 How to read this

This is a reference manual, not a task list. Implementation order is dictated by the Kanban board, and each ticket maps to a specific entry (e.g. [3.1.2]) so the work matches the design. Sections 1–5 are context and contracts that apply across the whole system; section 6 is the per-operation detail.

1.3 Scope

In scope for v0.1:

- Boards, with per-board columns (status lanes)
- Tickets, including nested boards (a ticket can contain a board)
- Comments, including threaded replies
- Tags, global and board-local, with a "special" flag
- Users, teams, and team membership with roles
- Ticket assignment and tagging
- Local-first operation: the app runs fully on a local store with no account required (2.1)

Out of scope for v0.1 (revisit later): cross-device team synchronisation (the local/server split is designed for it now, but the sync engine itself comes later — see 3.5), real-time collaboration, notifications, file attachments, search, and any mobile-specific client.

1.4 Goals & non-goals

Goals: a clean separation between frontend and backend that lets the two be built in parallel; a normalised data model with referential integrity enforced by the database; permission rules enforced server-side.

Non-goals: this is not a general project-management suite (no Gantt charts, time tracking, or billing), and it does not aim to be a real-time multiplayer editor in this revision.

1.5 Glossary & abbreviations

Board — a container of columns and tickets.

Column — a status lane within a board (e.g. To Do / Doing / Done); ordering is per board.

Ticket — a card on a board, sitting in exactly one column.

Sub-board — a board contained by a ticket (drilling into the ticket opens it).

Mother ticket — the inverse: the ticket that contains a given sub-board.

Comment — a note on a ticket; comments can reply to other comments.

Tag — a label applied to tickets; *global* (any board) or *board-local*.

Special tag — a tag flagged `SPECIAL = true`; same table, distinguished by the flag.

Team — a group of users.

Role — a user's capability level within a team (owner / admin / member).

Local mode — the app running against its own local store with no account; the default, offline-capable mode.

Team sync — replicating a team's boards between a client and the server; requires an account and team membership (planned, see 3.5).

FK — foreign key; **PK** — primary key; **Composite PK** — a key spanning two columns.

1.6 Actors & roles

Roles are defined per team (see `team_members.ROLE`):

- **Owner** — full control of the team and its boards; manages members and roles.
- **Admin** — manages boards, columns, tickets, and tags; cannot remove the owner.
- **Member** — works within boards: creates and edits tickets, comments, and assignments.

A user may belong to several teams and hold a different role in each. The exact capability-to-role mapping is defined in 4.5.

2. System overview

2.1 Architecture

Three tiers: client $\rightarrow$ backend $\rightarrow$ database. The split holds in both deployment modes; only where the backend and database run changes.

The client never talks to the database directly and never holds database credentials. The Go backend owns every query and is where all permission checks run, so a client cannot bypass them. The client always reaches the backend the same way — a websocket connection to a host and port — and is deliberately ignorant of whether that backend is local or remote. Only the connection target differs, which is what lets the client be written once.

Local mode (default). The backend runs in-process inside the same binary as a goroutine, listening on a loopback websocket (127.0.0.1), and persists to a local SQLite store. No account is required and no network is used. This is the mode an end user gets by simply running the app — there is no separate process to supervise and nothing to orphan if the app exits.

Server (team) mode. The same backend runs as a deployed service against PostgreSQL and accepts websocket connections from remote clients. A client joins a team by authenticating against its server (4.1); membership and roles then gate every operation.

The backend is written once as well: it reaches its database through a single storage interface (the **Store**, 2.4) with two implementations — SQLite for local, PostgreSQL for server — so business logic and permission checks live in one place and only the data-access layer differs. PostgreSQL (server) and SQLite (local) are each the system of record for their own data, with referential integrity enforced by foreign keys and check constraints (section 3).

2.2 Tech stack & dependencies

- **Backend:** Go
- **Databases:** PostgreSQL (server) and SQLite (local)
- **SQLite driver:** modernc.org/sqlite — pure Go, no CGO, so the local build stays a single statically-linked cross-platform binary
- **PostgreSQL driver:** `pgx`
- **Data-access layer:** a hand-written **Store** interface with two implementations (2.4); no ORM
- **IDs:** github.com/google/uuid (UUID primary keys) — also the offline ID-generation strategy: clients mint IDs locally without server coordination, which is what lets local and synced rows share one namespace
- **Client transport:** websocket (same protocol local and remote); *TODO* — *confirm websocket library and whether any plain HTTP endpoints are also exposed*
- **Frontend:** `svelte5`

2.3 Environments & configuration

Server mode uses three environments — development, staging, production — sharing one schema. Server configuration (database DSN, listen address, secrets) is supplied through environment variables; no secrets live in the repository. *TODO: list the required variables once finalised.*

Local mode is self-configuring: it needs no DSN or secrets and starts its own backend automatically. The loopback listen port defaults to an OS-assigned ephemeral port (bind `:0`, then hand the chosen port to the in-process client) so it never collides with other software or a second instance; a fixed port is available only as a developer-settings override, not the shipping default. A Unix domain socket may be used instead of TCP to remove the listening network port altogether (6.1).

2.4 Storage abstraction & distribution

All database access goes through one `Store` interface. The business logic depends only on that interface; the two implementations — SQLite (local) and PostgreSQL (server) — are the only place the two backends diverge. The interface is introduced from the start and every query routes through it, so neither backend's SQL dialect leaks into the rest of the codebase; retrofitting this seam after Postgres-specific SQL has spread is the expensive path and is avoided.

SQLite-only and PostgreSQL-only differences are absorbed inside their respective implementations. Where the local store needs a capability PostgreSQL provides natively, it is implemented in application code against SQLite rather than pushed back into the shared layer.

Distribution. End users install a single statically-linked binary (backend plus embedded SQLite, no CGO, no external services). They never run a database or a container — local mode spawns its own backend, so there is no Docker or PostgreSQL for a normal user to manage. Server deployments run the same codebase as a separate service alongside a managed PostgreSQL instance; there, backend and database are distinct units.

3 Data model

the database — persisted source of truth (see the table CSVs)

Conventions used throughout: primary keys are UUIDs; timestamps are `TIMESTAMPTZ` (timezone-aware); column names are upper snake case. A column with no `NOT NULL` is nullable. Where a nullable FK has a meaning attached to `NULL`, it is stated in that table's `notes`.

3.1 Shared enums & types

Team role (`team_members.ROLE`, `TEXT`): owner, admin, member. Defaults to member.

Tag colour (`tags.COLOUR`, `TEXT`): a hex string matching `^[0-9A-Fa-f]{6}$`, e.g. `#1D9E75`.

Status / columns: status is *not* an enum — it is modelled as rows in the columns table so each board defines its own lanes (3.2.3).

3.2 Tables

3.2.1 boards

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		
DESCRIPTION	TEXT	CHECK(LENGTH(DESCRIPTION) < 10000)		
AUTHOR	UUID	NOT NULL. REFERENCES users(ID) ON DELETE RESTRICT		
CREATION	TIMESTAMPZ	NOT NULL. DEFAULT now()		
MOTHER_TICKET	UUID	REFERENCES tickets(ID) ON DELETE CASCADE	Ticket that contains this board	null = top-level board

3.2.2 tickets

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		
DESCRIPTION	TEXT	CHECK(LENGTH(DESCRIPTION) < 10000)		
AUTHOR	UUID	NOT NULL. REFERENCES users(ID) ON DELETE RESTRICT		
CREATION	TIMESTAMPZ	NOT NULL. DEFAULT now()		
DUE_DATE	TIMESTAMPZ			
BOARD_ID	UUID	NOT NULL. REFERENCES boards(ID) ON DELETE CASCADE	Board the ticket lives on	
COLUMN_ID	UUID	NOT NULL. REFERENCES columns(ID) ON DELETE RESTRICT		Added: was missing from the struct
SUB_BOARD	UUID	REFERENCES boards(ID) ON DELETE SET NULL	Board this ticket contains	Add UNIQUE if one board has a single owner ticket

3.2.3 columns

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
BOARD_ID	UUID	NOT NULL. REFERENCES boards(ID) ON DELETE CASCADE		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		
POSITION	INTEGER	NOT NULL. CHECK(POSITION >= 0)		0-based ordering within the board

3.2.4 comments

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
AUTHOR	UUID	NOT NULL. REFERENCES users(ID) ON DELETE RESTRICT		
CREATION	TIMESTAMPZ	NOT NULL. DEFAULT now()		
CONTENT	TEXT	NOT NULL. CHECK(LENGTH(CONTENT) < 10000)		
TICKET_ID	UUID	NOT NULL. REFERENCES tickets(ID) ON DELETE CASCADE		
PARENT_ID	UUID	REFERENCES comments(ID) ON DELETE CASCADE	Parent comment when this is a reply	null = top-level comment

3.2.5 tags

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		
DESCRIPTION	TEXT	CHECK(LENGTH(DESCRIPTION) < 10000)		
COLOUR	TEXT	NOT NULL. CHECK(COLOUR ~ '^[0-9A-Fa-f]{6}\$')		e.g. <code>\#1D9E75</code>
SPECIAL	BOOLEAN	NOT NULL. DEFAULT false		
BOARD_ID	UUID	REFERENCES boards(ID) ON DELETE CASCADE		null = global tag

3.2.6 users

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		

3.2.7 teams

name	type	constraints	description	notes
ID	UUID	PRIMARY KEY		
NAME	TEXT	NOT NULL. CHECK(LENGTH(NAME) < 1000)		

3.2.8 ticket_tags

name	type	constraints	description	notes
TICKET_ID	UUID	NOT NULL. REFERENCES tickets(ID) ON DELETE CASCADE		Composite PK (TICKET_ID + TAG_ID)
TAG_ID	UUID	NOT NULL. REFERENCES tags(ID) ON DELETE CASCADE		Composite PK (TICKET_ID + TAG_ID)

3.2.9 ticket_assignee

name	type	constraints	description	notes
TICKET_ID	UUID	NOT NULL. REFERENCES tickets(ID) ON DELETE CASCADE		Composite PK (TICKET_ID + USER_ID)
USER_ID	UUID	NOT NULL. REFERENCES users(ID) ON DELETE CASCADE		Composite PK (TICKET_ID + USER_ID)

3.2.10 team_members

name	type	constraints	description	notes
TEAM_ID	UUID	NOT NULL. REFERENCES teams(ID) ON DELETE CASCADE		Composite PK (TEAM_ID + USER_ID)
USER_ID	UUID	NOT NULL. REFERENCES users(ID) ON DELETE CASCADE		Composite PK (TEAM_ID + USER_ID)
ROLE	TEXT	NOT NULL. DEFAULT 'member'		e.g. owner / admin / member

3.4 Relationships overview

One-to-many (FK on the child): a board has many columns and many tickets; a column holds many tickets; a ticket has many comments. Comments are self-referential — a comment may have a parent comment, forming reply threads.

Many-to-many (via join tables): tickets and tags through `ticket_tags`; tickets and users (assignment) through `ticket_assignee`; users and teams through `team_members`, which also carries the role.

The board–ticket nesting is circular: a ticket points down into a board (`tickets.SUB_BOARD`) and a board points back up to its containing ticket (`boards.MOTHER_TICKET`). Because the two tables reference each other, create both tables first and add the foreign keys afterwards (or mark them DEFERRABLE); neither can reference a table that doesn't yet exist.

3.5 Local mode & future synchronisation

Forward-looking — not implemented in v0.1. Recorded here so the foundation is laid correctly now, since retrofitting sync later is far more expensive. The finalised schema in 3.2 is not changed by this section.

Two areas need a decision before team sync is built.

Account-less authorship. In local mode there is no authenticated user, yet `boards.AUTHOR`, `tickets.AUTHOR`, and `comments.AUTHOR` are NOT NULL REFERENCES `users(ID)`. Local mode therefore needs a synthetic local user row

Sync scope and metadata. The unit of synchronisation is a team and the board subtree it owns (`board` $\rightarrow$ `columns` $\rightarrow$ `tickets` $\rightarrow$ `comments` / `tags` / `assignees`). A board is local-only or team-owned; marking that distinction (e.g. a nullable team reference on `boards`, NULL = local) plus per-row change tracking (`updated_at`, a tombstone/soft-delete column so deletes propagate, and a revision or dirty flag for push) is what sync will require. UUID primary keys already let local and server rows share one namespace without collisions, and last-write-wins on `updated_at` is the cheapest starting conflict policy.

One consequence to watch: the deliberate double-stored board/ticket nesting (`tickets.SUB_BOARD` and `boards.MOTHER_TICKET`, 3.4) becomes a distributed invariant once rows sync — a merge must keep both sides consistent. This is a known cost of the chosen design, not a reason to change it.

4 API conventions

These apply to every operation in section 6 so individual entries don't repeat them.

4.1 Auth & sessions

Authentication is a team concern only. Local mode runs with no account: the user is never asked to log in, and the backend serves every local request without resolving a user. An account is required only to join a team and act on its boards — at that point the client authenticates against the team's server, receives a session token, and that token accompanies every subsequent request so the backend can resolve the current user before doing anything else. Passwords are stored hashed, never in plaintext. *TODO: confirm token mechanism (opaque session token vs JWT) and expiry policy.*

Separately from user auth, the local loopback backend issues a per-session token to its in-process client at startup and rejects callers that do not present it (6.1). This guards the local store even though there is no login: no account is not the same as no gate.

4.2 Request / response format

Requests and responses are JSON over HTTP. A successful response returns the relevant resource (or a status line); a failure returns an error code from section 7.1 together with a human-readable message. The shapes of resources follow the structs in section 5.

4.3 Pagination, filtering, sorting

List endpoints page their results rather than returning everything. *TODO: settle on limit/offset or cursor-based paging, and define the common filter and sort parameters.*

4.4 Versioning

The API is versioned in the path (e.g. `/api/v1/...`). Breaking changes bump the version; additive changes do not. The document revision (section 0) tracks spec changes independently.

4.5 Permission model

Every operation declares the role it requires. The backend enforces this after resolving the current user (4.1) — permission checks never run on the client. Checks combine team role with scope: a user only acts on boards belonging to a team they are a member of, at the role that operation requires. In local mode there is no team and no role to resolve: the local user has full capability over

local boards, so the matrix below applies only to team-owned boards in server mode.

Capability	Member	Admin	Owner
View boards / tickets / comments	Y	Y	Y
Create / edit tickets, comments	Y	Y	Y
Manage columns, tags, boards		Y	Y
Manage members and roles			Y

Draft — adjust the rows to match the operations you settle on in section 6.

4.6 Error handling

All failures resolve to a code from section 7.1, returned as `code | message`. Operations list the specific codes they can return in their `Errors` field.

5 Models

the Go structs over the schema (source of truth: models.go)

5.1 Structs

5.1.1 Board

```
type Board struct {
    ID          uuid.UUID
    Name        string
    Description  string
    Author      uuid.UUID // user
    Creation    time.Time
    Tags        []Tag
    MotherTicket *uuid.UUID // ticket
    Tickets     []Ticket
}
```

5.1.2 Ticket

```
type Ticket struct {
    ID          uuid.UUID
    Name        string
    Description  string
    Comments    []Comment
    Author      uuid.UUID // user
    Creation    time.Time
    DueDate     time.Time
}
```

```
Tags      []Tag
BoardID    uuid.UUID // mother board
SubBoard   *uuid.UUID // board
Assigned   []User
ColumnID   uuid.UUID
}
```

5.1.3 Comment

```
type Comment struct {
    ID          uuid.UUID
    Author      uuid.UUID // user
    Creation    time.Time
    Content     string
    Replies     []Comment
    TicketID    uuid.UUID // mother ticket
    ParentID    *uuid.UUID // parent comment
}
```

5.1.4 User

```
type User struct {
    ID          uuid.UUID
    Name        string
    Teams       []Team
    Assigned    []Ticket
}
```

5.1.5 Tag

```
type Tag struct {
    ID          uuid.UUID
    Name        string
    Description  string
    Colour      string // hex
    Special     bool
    BoardID     *uuid.UUID // board, nil = global
}
```

5.1.6 Team

```
type Team struct {
    ID          uuid.UUID
    Name        string
    Members     []User
}
```

5.1.7 Column

```
type Column struct {  
    ID        uuid.UUID  
    BoardID   uuid.UUID // board  
    Name      string  
    Position  int  
}
```

5.1.8 TicketTag

```
type TicketTag struct {  
    TicketID uuid.UUID // ticket  
    TagID    uuid.UUID // tag  
}
```

5.1.9 Ticket Assignee

```
type TicketAssignee struct {  
    TicketID uuid.UUID // ticket  
    UserID   uuid.UUID // user  
}
```

5.1.10 Team Member

```
type TeamMember struct {  
    TeamID uuid.UUID // team  
    UserID uuid.UUID // user  
    Role   string  
}
```

5.2 Struct to table mapping

The structs are not a one-to-one mirror of the tables — some fields are columns, others are relationships the backend loads.

Scalar fields map directly to columns (`ID`, `Name`, `BoardID`, `Creation`, ...).

Slice fields are loaded relationships, not columns: `Board.Tickets`, `Board.Tags`, `Ticket.Comments`, `Ticket.Assigned`, `Comment.Replies`, `User.Teams`, `Team.Members`. They are populated by querying the relevant FK or join table, never stored on the parent row. The many-to-many slices are backed by the join tables — `Tags` by `ticket_tags`, `Assigned` by `ticket_assignee`, `Teams/Members` by `team_members`.

Pointer fields (`*uuid.UUID`) are the nullable foreign keys: `Board.MotherTicket`, `Ticket.SubBoard`, `Comment.ParentID`, and `Tag.BoardID`. A `nil` means the relationship is absent (top-level board, no sub-board, top-level comment, global tag).

Loaded relationships are filled lazily. This matters most for the board–ticket nesting: loading a board could otherwise cascade into its tickets, their sub-boards, and so on, so the backend only loads one level at a time.

6 Non-functional requirements

6.1 Security

Permissions are enforced server-side only (4.5); clients hold no database credentials. Passwords are hashed at rest. Inputs are validated against the constraints in section 3 before they reach the database. Remote (team) traffic is over TLS.

Local listener hardening. The local backend listens on a loopback socket with no user authentication, so the transport itself is the gate. It binds to `127.0.0.1` only (never `0.0.0.0`), checks the `Origin` header on the websocket upgrade and rejects anything unexpected, and requires the per-session token it handed its client at startup (4.1). Together these defend against other local processes and against browser-based cross-origin / DNS-rebinding attacks that target open localhost servers. A Unix domain socket may be used instead, removing the network port and the cross-origin surface entirely.

At-rest encryption (optional). The local store is unencrypted by default; for the lost-or-stolen-device case, OS full-disk encryption already covers it. Encryption is offered opt-in: the preferred form derives a key from the OS keychain (Keychain / Credential Manager / Secret Service) so the store is encrypted with no password prompt, with a user-chosen passphrase available as a stronger opt-in vault. SQLCipher is deliberately avoided because it requires CGO and would break the single statically-linked binary; if field-level encryption is wanted later, it is done in application code against the pure-Go driver. *TODO: decide whether to ship keychain-derived encryption in v0.1 or defer it.*

6.2 Privacy & data protection

Deleting a user is blocked while they still author boards, tickets, or comments (`ON DELETE RESTRICT`); reassign or handle that content first. Deleting a board cascades to its columns, tickets, and the tickets' comments. *TODO: define a data-retention and account-deletion policy.*

6.3 Performance & scalability

Index foreign keys that drive list queries (`tickets.BOARD_ID`, `tickets.COLUMN_ID`, the join-table columns). Load a board's tickets with a single `WHERE board_id = ?` rather than per-ticket lookups, to avoid the N+1 pattern. List endpoints page their results (4.3). In local mode the loopback websocket serialises every call rather than invoking in-process functions directly; for this workload that overhead is negligible and is accepted in exchange for a single shared client and backend codebase. *TODO: set concrete latency and throughput targets.*

6.4 Reliability

backups, availability

Regular automated database backups with a defined restore procedure. *TODO: state the backup cadence, retention window, and any availability target.*

6.5 Accessibility & i18n

TODO: state the accessibility standard the frontend targets and whether localisation is in scope.

7 Cross-cutting

7.1 Error codes

Errors are returned as `code | message`. 0 is success; everything else is a failure. *Draft — extend as operations are written.*

Code	Meaning
0	Success
1	Bad request / invalid input
2	Unauthenticated
3	Forbidden (insufficient role)
4	Not found
5	Conflict (constraint violation)
9	Internal error

7.2 Logging & monitoring

The backend emits structured logs for every request (who, what, outcome). Mutations to boards, tickets, and permissions are logged so changes are traceable.
TODO: choose a log format and a monitoring/alerting destination.